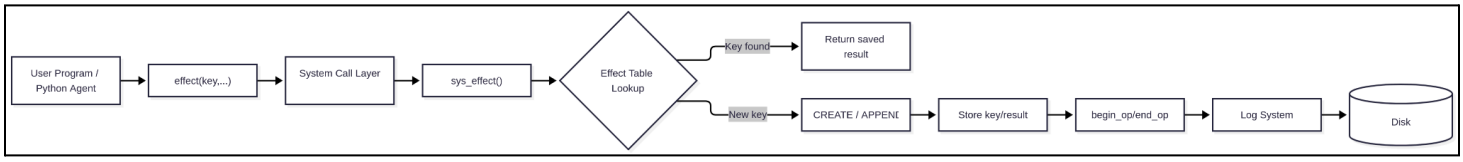
Exactly-once syscalls across crashes

Operating Systems Project Report by Group 4 (S. O. S.)

Architecture

The project extends xv6-public (x86) with an idempotent filesystem operation. A user program sends a request containing a 64-bit key, operation, file path and payload. The kernel checks the persistent effect table before executing the operation. If the key already exists with the same request, the stored result is returned. Otherwise, the requested file operation and its effect record are handled through the filesystem transaction mechanism.



Syscall implementation

We implemented the exactly-once syscall mechanism by introducing a **custom effect() system call** supporting idempotent CREATE and APPEND operations.
Each request is identified by a **unique 64-bit key,** with FNV-1a* hashing used to validate request consistency and detect conflicting reuse of keys. We integrated a **persistent-effect record table**, using hash-based open addressing and linear probing to store request keys, request hashes and execution results. We also added a **sync-lock** which serializes concurrent requests, while the filesystem modification and corresponding effect-record insertion are performed within a single logging transaction to ensure atomicity across crashes. We also implemented a **crashat() system call** to inject failures at different stages of the filesystem commit process and verify recovery behaviour.

Disk layout and effect table

Persistent Effect Table:
We implemented a persistent effect table that stores request keys, results, and hashes. We reserved two disk blocks using mkfs.c and the superblock, supporting 64 records that survive system restarts. We also implemented effect_lookup() and effect_insert().
Atomic File Operations:
We implemented effect_apply() for CREATE and APPEND. Using xv6's logging system, file modifications and effect-table updates occur in one transaction. A sleep lock prevents duplicate concurrent execution.
Crash Injection and Recovery:
We implemented crashat(n) to simulate crashes at different logging stages. Testing confirmed atomic recovery after crashes and that retrying completed requests does not duplicate file modifications.

Bridge connection and communication

We implemented a Python QEMU bridge that provides communication between the external agent and the xv6 guest. It connects to the TCP serial interface at 127.0.0.1:5555, sends shell commands and reads the guest's responses. The bridge includes command execution and file-operation support, along with timeout and connection-error handling. This provides an interface for interacting with the filesystem without manually typing every command into the QEMU terminal.

Agent/validator

The Python agent sends each user request to the hosted LLM, which returns a structured tool proposal. The validator checks that the proposed tool is allowlisted and that its arguments have the expected types and safe paths before dispatching it to the QEMU bridge. The agent records tool results as context for later requests. This keeps model output separate from execution: the LLM proposes an action, while the validator controls which requests may reach xv6. The bridge handles communication with the guest OS.

Challenges

One major challenge was ensuring that the file modification and the corresponding effect-table record remain consistent after a crash. Another challenge was supporting 64-bit keys in 32-bit xv6. Concurrent requests also required synchronization to prevent duplicate execution. Crash testing introduced additional complexity because the filesystem image must be preserved between a crash and replay. Correctly distinguishing log-write crash points from commit-stage checkpoints is another important implementation concern.

Innovations

The project reuses xv6's existing filesystem logging mechanism instead of introducing a separate transaction system. A persistent idempotency table allows the kernel to recognize previously processed requests. Request information is also checked to detect conflicting reuse of a key. Crash-injection hooks provide a way to investigate filesystem recovery behavior. The Python bridge and validator connect an external agent to the operating-system guest through a controlled communication interface.

Github repo link: https://github.com/Rahul-dev2701/os-project